Термино

SYSTEM ARCHITECTURE & OPERATING GUIDE

A practical guide to how the application works, how its parts connect, and how it protects shared clinic work.

One system. Many clinics. Coordinated care.

Термино is a medical appointment scheduling platform developed by CODEX Computers. It combines a public doctor and clinic directory with protected patient, doctor and reception workspaces, and a separate company administration area.

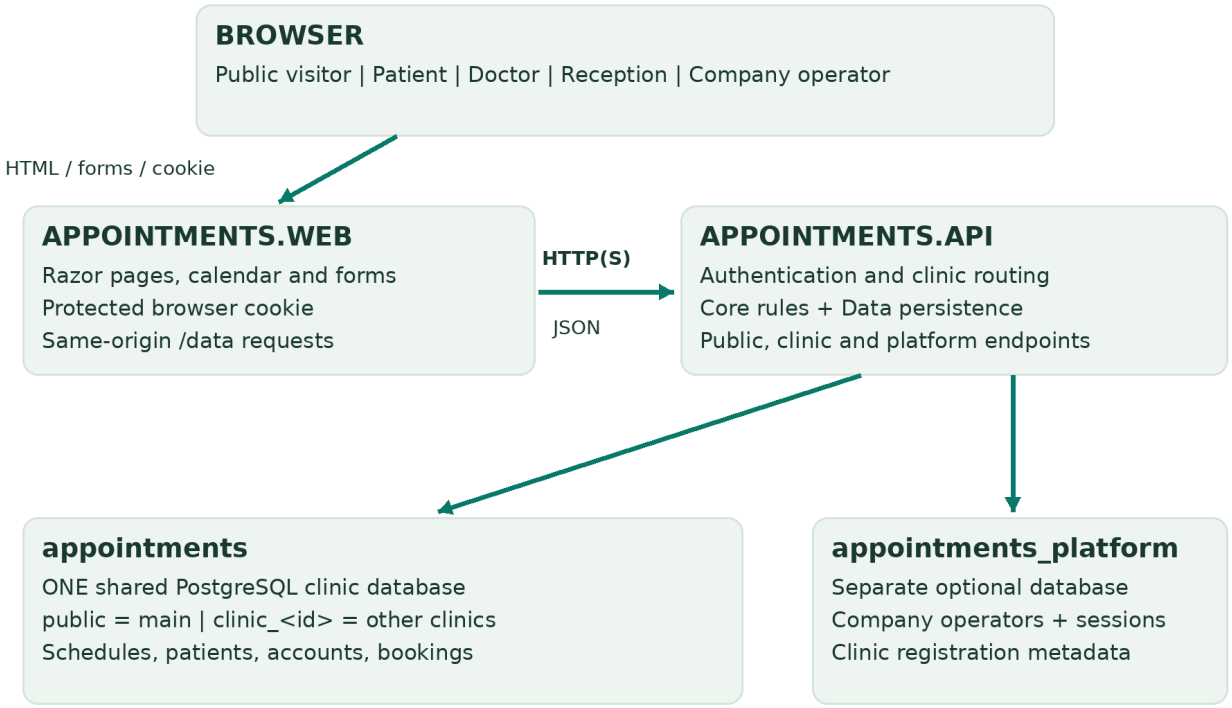
At a glance	Current implementation
Application	ASP.NET Core MVC Web + separate ASP.NET Core API
Runtime	C# / .NET 10; PostgreSQL 17
Clinic storage	One appointments database; one schema per clinic
Company storage	Optional separate appointments_platform database
Interface	Macedonian (mk-MK); user-facing brand Термино
Concurrency	Database transactions, advisory locks, overlap constraints, versions and idempotency

DOCUMENT SCOPE
This guide describes repository source at commit 1a91629, reviewed on 20 October 2026. It does not verify the contents, credentials or deployed version of the live server. Careline identifiers remain in internal code and configuration.

Prepared in English, with Macedonian interface labels where useful. Intended for owners, developers, support staff and anyone taking over the system. Contains no passwords, tokens or patient data.

01 The system in one picture

Start here: the browser talks to Web, Web talks to API, and API owns database access.



Only the API connects to PostgreSQL. Public responses exclude private records.

Figure 1. Logical connections. Web and API can be hosted separately; the diagram describes responsibilities, not physical server count.

Web renders the interface and handles browser forms. The API authenticates users, chooses the clinic, checks permissions and invokes domain rules. The Data layer commits changes to PostgreSQL. Contracts define the JSON messages shared by Web and API.

Read next	Pages
Technologies and database design	3–5
Roles, user journey and appointment rules	6–8
Concurrency, security and integrations	9–11
Provisioning, deployment and verification	12–14
Glossary and source map	15

TWO DIFFERENT BOUNDARIES

A clinic administrator runs one clinic. A company/platform operator registers and manages clinics. Company access is a separate identity boundary and does not automatically grant access to clinic appointments.

02 Technologies and code structure

The application is a .NET solution with clear responsibilities and a lightweight browser interface.

Technology	Purpose
C# / .NET 10	Application runtime and business logic; all projects target net10.0.
ASP.NET Core MVC + Razor	Server-rendered pages, protected browser session and same-origin proxy actions.
ASP.NET Core Web API	JSON endpoints, authorization, clinic routing and account lifecycle.
Entity Framework Core 10.0.12	Relational mappings, migrations, tracked updates and concurrency tokens.
Npgsql EF provider 10.0.3	PostgreSQL connectivity and EF integration.
PostgreSQL 17 + btree_gist	Durable data, transactions, indexes and interval-overlap constraints.
JavaScript / CSS / HTML	Interactive forms, searchable selects, responsive layout and subtle motion.
FullCalendar Standard 6.1.20	Locally bundled calendar: day, week, month and validated drag rescheduling.
IIS / ASP.NET Core hosting	The described Windows deployment; Kestrel also supports development hosting.

Solution projects

Project	Responsibility
Appointments.Web	Views, static assets, browser authentication cookie and ApiClient.
Appointments.Api	HTTP boundaries, identity, clinic registry and company provisioning.
Appointments.Core	Framework-independent scheduling, status, catalogue and reporting rules.
Appointments.Data	EF mappings/migrations; PostgreSQL transactions, locks and audit writes.
Appointments.Contracts	Shared request/response records and input validation.

Node.js, Python, jsdom, Playwright and Docker are development/test tools, not required browser services. Native PostgreSQL works without Docker. There is no frontend compilation step or paid FullCalendar plugin requirement. GitHub hosts source and CI; it is not the application database.

03 One database, many clinic schemas

Multi-tenancy means many clinics share the application while their operational records remain separated.

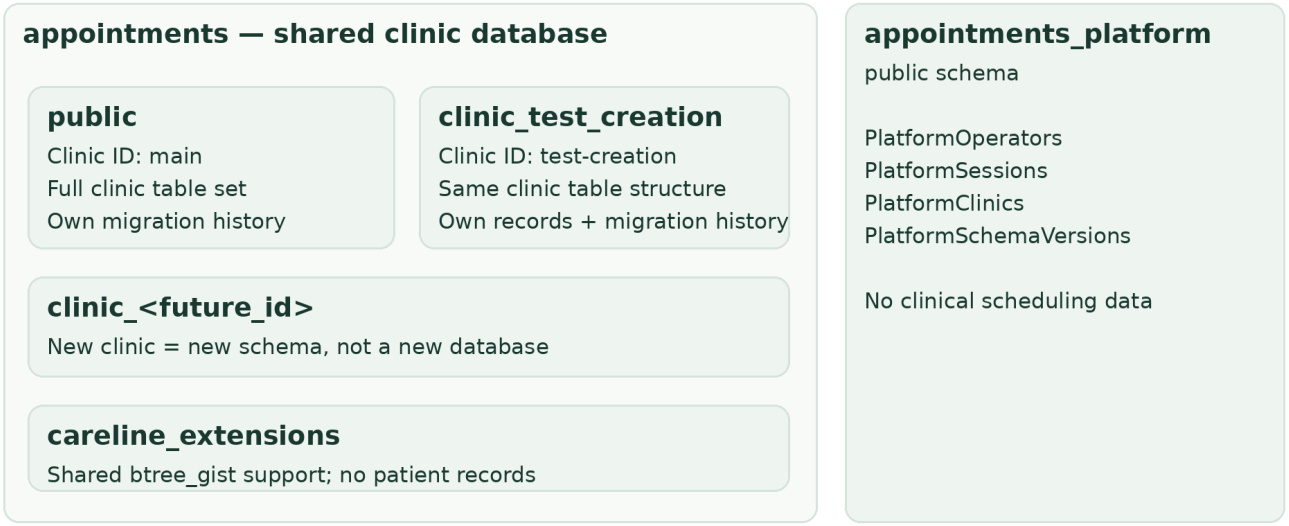


Figure 2. Logical storage. The test clinic schema is illustrative; actual schema names derive from the immutable clinic ID.

A PostgreSQL database is a storage boundary. A schema is a named namespace inside it, similar to a folder containing tables. Each clinic has its own Doctors, Patients, Appointments, Accounts and related tables. This is schema-per-tenant storage, rather than one shared table filtered by ClinicId.

- The original main clinic uses public. It has the same business model as other clinics; public is its historical schema name, not a public-access permission.
- Other clinic IDs map deterministically: north-clinic → clinic_north_clinic. Underscores are forbidden in clinic IDs to avoid mapping collisions.
- API routing sets the connection search path to the selected clinic schema plus careline_extensions. Other clinics never fall back to public. A fully qualified Settings.ClinicId check rejects a mismatched or missing binding.

ISOLATION AND SHARED RISK

Browser accounts cannot choose arbitrary schemas or connection strings. The trusted application database login can access multiple schemas; schemas are not independent database-user security sandboxes. One shared database outage affects all clinics, so backup and restore planning must cover the whole database.

04 The data model

A patient profile is scheduling identity/contact information; an account is permission to sign in.

Records	Meaning and connections
Doctors / WorkingPeriods / SourceSchedules	Doctor identity, current weekly hours and imported source references. Doctors can require confirmation or reception-only booking.
Specialties / Services / DoctorServices	Catalogue. A service belongs to a specialty; assignments connect services and doctors. Service duration determines appointment length.
Patients	Scheduling name and contact details. Not an electronic medical record, diagnosis or treatment chart.
Appointments	References one doctor and patient; optional service ID/name snapshot; UTC start/end, status, version and creator/request identity.
AvailabilityExceptions	One-off available intervals or blocked periods for one doctor/date. Blocks take precedence over added hours.
Accounts / Sessions	Role and optional profile link; password hash and access state. Sessions contain token hashes and expiry, not raw bearer tokens.
AppointmentHistory	Before/after booking times and statuses, actor name/role snapshots, timestamp and appointment version.
IdempotencyRequests / AuditEvents	Safe booking replay; operational actor/action/target/time records.
Settings / EF migration history	Clinic binding, time zone, and schema evolution tracking.

Important relationships

Doctor 1 → many appointments; patient 1 → many appointments. Doctors and services have many-to-many assignments through DoctorServices. A patient or real doctor profile can have at most one linked account. An administrator has no patient/doctor profile link. Sessions belong to accounts; history entries belong to appointments.

Foreign keys, unique indexes and check constraints enforce references, valid roles/statuses, positive versions and sensible time ranges. Identical IDs in different clinic schemas remain separate records. Source service-only profiles are not real doctors and cannot receive doctor-role login accounts.

HISTORY IS PURPOSE-SPECIFIC

Appointment history is readable operational history. Generic audit events provide a broader activity trail. Neither is claimed to be immutable or tamper-proof compliance storage; historical events that predate the feature are not invented or backfilled.

05 Roles, profiles and clinic access

Permissions are enforced by the API, independently of which menu items the browser shows.

Role	Typical scope
Visitor	Search public doctors/clinics, view public services and read-only free times. No patient records, account data or booking mutations.
Patient	Register/sign in for a clinic; book own visits, view own history, reschedule or cancel eligible upcoming appointments.
Doctor	Own appointments/calendar and availability; authorized patient scheduling workflows. Cannot edit another doctor’s schedule or administer accounts/catalogue.
Administrator / receptionist	Clinic-wide scheduling, patients, doctor/service catalogue, hours/exceptions, account access and reports. The receptionist workspace uses the Administrator application role.
Platform operator	Separate company session: register/list clinics and manage company credentials. Not an automatic clinic session.

Profiles and accounts are different

Reception may create a patient profile during a phone booking before that person has a login. Later, after verifying identity, reception can create an account linked to that existing profile. Self-registration creates a new patient; matching an email does not let someone claim an existing profile. Disabling an account removes access without deleting its appointments or profile.

Clinic membership is explicit

A clinic account belongs to one clinic. Login resolves the clinic from a globally unique sign-in email; the browser cannot use a clinic hint to change account ownership. Public discovery spans clinics, but an account from clinic A cannot book in clinic B through the same membership. A global patient identity with multiple clinic memberships is not implemented.

Authenticated API requests carry the session token and clinic context. Web forwards the protected clinic identity and requires a matching X-Workspace-Clinic header for workspace data requests. This prevents an old tab from silently operating in a clinic selected in another tab.

06 From discovery to a saved booking

The public journey informs visitors; authenticated booking creates the reservation.

1. Discover and compare

The landing page searches doctor names, clinic names, specialties/subspecialties and active services. Broad search matches any of those fields; optional filters combine with AND. Explicit Cyrillic case folding supports Macedonian names even on PostgreSQL C/POSIX locales. Search does not transliterate Latin/Cyrillic or correct misspellings.

2. Preview a doctor’s availability

The profile displays date/service filters, seven quick day links and free times, without requiring sign-in. It uses the same hours, booked intervals, absences, service duration, time zone and release rules as booking. No schedule, reception-only access, unavailable services and fully booked days have corresponding messages. These times are information, not reservations.

3. Continue through sign-in

The selected doctor and clinic survive the login/registration return URL. Web permits local return URLs and checks clinic membership before opening the booking dialog. The visitor must sign in or create an eligible patient account to book.

4. Select, review and save

Choose doctor, service where configured, date and an authoritative available slot. Staff can select/create a patient; a patient books only for themselves. Review creates no reservation. Submission sends the intended booking plus a unique request ID. The server repeats all permissions, duration, availability and overlap checks before saving.

5. Record the result

A normal booking is Confirmed immediately unless the doctor requires confirmation; then it starts Scheduled (Чека потврда). The appointment, creation history, audit event and replay record are saved together. Other authorized users see the shared result on refresh.

NO FALSE PROMISE OF A HELD SLOT

Two people can view the same free time. Only a successful server save reserves it. If another user takes it first, the later save returns a conflict and the user chooses another time. No automatic email or SMS is implied by confirmation.

07 Scheduling rules and lifecycle

Stored timestamps are UTC instants; users choose and see local clinic time.

Weekly periods define ordinary hours, including split shifts. Dated exceptions add one-off availability or block time. Services define duration (10–120 minutes in five-minute steps); legacy bookings without a service use the doctor’s configured duration. The booking horizon is today through 180 days ahead. Ambiguous/nonexistent daylight-saving times and invalid duration transitions are excluded.

Status / UI label	How it is assigned
Scheduled / Чека потврда	Normal booking or rescheduling when the doctor requires confirmation. Staff may confirm it before its start.
Confirmed / Потврден	Immediate normal booking where confirmation is not required; staff confirmation of a future pending request; reception’s agreed-time booking.
Cancelled / Откажан	An authorized user cancels an eligible upcoming booking. Staff can also cancel an overdue unconfirmed request. Releases its occupied interval.
Completed / Завршен	Reception or the appointment’s doctor records completion after the appointment ends; accepts Confirmed or overdue Scheduled.
NoShow / Не се појавил	Reception or the appointment’s doctor records non-attendance after the appointment ends; accepts Confirmed or overdue Scheduled.

Special cases that matter

- Reception may enter a time agreed directly with the doctor, even without weekly hours. This creates an exact one-off interval and a Confirmed appointment atomically; it still cannot override an absence, overlap, invalid time or booking horizon.
- Changing schedule/absence rules cannot strand active future or ongoing bookings. The Tuesday-first policy can hold Wednesday slots until Tuesday is full for configured doctors.
- Rescheduling uses normal available slots, starts on the original local appointment date and re-evaluates the confirmation setting. Terminal appointments remain closed.
- Overdue pending requests have an explicit staff resolution path; they do not expire or become confirmed automatically. Patients cannot resolve overdue requests.

08 Concurrency and conflict resolution

Several receptionists can work in the same clinic. The database is the final authority.

Protection	What it prevents
Transaction: all-or-nothing save	An appointment cannot be committed without its related history/audit/replay writes. Validation or database failure rolls the transaction back.
Transaction advisory locks	Serialize competing operations for the same doctor and coordinate catalogue changes. Lock keys include the clinic schema; they work across API instances.
GiST exclusion constraints	Reject overlapping occupied intervals for the same doctor OR patient. Protect patient races even when two different doctor locks are involved.
Optimistic version numbers	Reject stale appointment, schedule, catalogue and patient edits with HTTP 409 instead of silently overwriting newer data.
Booking idempotency	Same account + request ID + identical command returns the original response. Reusing the key with different data is a conflict.
UI submission guards	Disable pending form submissions. Duplicate exception checks also protect repeated submissions at the server.

Example: two receptionists book 14:00

Both see the time as free. A saves first. B’s save runs after the competing work and checks current data; it is rejected as a conflict. If the same patient is being booked with different doctors, the patient overlap constraint provides the final protection. Adjacent 14:00–14:30 and 14:30–15:00 visits are allowed because intervals use [start, end).

Example: an older schedule form

A opens version 2. B changes hours and saves version 3. Background refresh may update A’s workspace data, but not the version attached to A’s displayed form. A’s stale save is rejected; their entries remain visible, with an explicit option to discard them and load current hours.

REFRESH IS CONVENIENCE; SAVING IS PROTECTED

Idle workspaces refresh about every 20 seconds and on focus, avoiding active editing/modal interruptions. This is polling, not instant push. A timeout can occur after a commit: refresh to inspect the result; mutation requests are not blindly retried.

09 Authentication and security boundaries

The browser does not receive database credentials or an API token in application JavaScript.

Browser → Web → API

Web stores the API token in a protected HttpOnly authentication cookie and forwards it server-to-server. SameSite and antiforgery protections apply to browser forms and workspace writes. Private API endpoints validate the live account/session, clinic, role and relevant doctor/patient ownership on every request.

Passwords and sessions

- Passwords use the ASP.NET Core versioned salted PBKDF2 hasher configured for 210,000 iterations. The original password cannot be recovered from its database hash; credentials can be reset through authorized account management.
- API sessions use random 48-byte opaque secrets; only SHA-256 token hashes are stored in the database. API session expiry is absolute at two hours. The Web cookie lifetime is separate and does not extend an expired API session.
- Password changes, credential provisioning and account disabling revoke relevant sessions. Five failed sign-ins lock an account for 15 minutes.
- Rate limits are per API process/IP: authentication 30 requests/minute; public discovery 90 requests/minute. A multi-instance deployment needs an appropriate gateway/distributed policy if shared limits are required.

Production configuration matters

Use HTTPS, correct host/proxy settings and protected persistent data-protection keys. The AllowHttpForTesting flag is an explicit intranet testing exception, not a production security target. Development demo identities are rejected outside Development; keep demo data and unrestricted demo access isolated from real operations.

CURRENT IDENTITY LIMITS

Email ownership verification, MFA, external identity federation and self-service email password recovery are not implemented. Operational audit tables are not a substitute for a defined privacy, retention, access-review and incident-response policy.

10 Interfaces, integrations and reporting

Most integration today is internal: the Web application consumes its own API.

Interface	Connection and purpose
Public discovery	GET /api/public/clinics and /api/public/doctors; public profiles and /availability. Explicit DTOs expose public metadata and free times only.
Workspace proxy	Browser calls same-origin /data actions. Web ApiClient forwards authenticated requests to the separate API.
Clinic operations	/api/auth, /api/bootstrap, /api/appointments, /api/slots, /api/patients, /api/schedule, /api/exceptions and catalogue/account routes.
Company operations	/api/platform routes use a separate platform session for clinic provisioning and company administration.
Health and API description	/health checks process readiness; authenticated /health/database checks database connectivity. OpenAPI is available in Development only.
Reports and export	Administrator-only /api/reports; on-demand aggregates with filters and CSV export. No external reporting service is required.

How to read report results

Reports use current appointment statuses and the appointment start date in the clinic’s time zone. Rescheduled visits appear on their current date. Distinct patient counts across rows are not additive. No-show percentage is no-shows divided by completed plus no-show visits. CSV export contains aggregates rather than patient contact lists.

Utilisation is an estimate reconstructed from current schedules, durations and saved exceptions. Historical schedule snapshots do not exist, so it is not a historical capacity audit. Reports use a consistent database read snapshot and must be requested again to refresh.

External services: what is and is not connected

The CODEX website and Facebook, LinkedIn and Instagram links are ordinary outbound footer links. There is no implemented SMS/email reminder provider, external calendar sync, payment gateway, hospital/EHR integration or external identity provider. The locally bundled calendar and CSS/SVG motion need no runtime CDN, paid scheduler or external API key.

11 Clinic provisioning and migrations

Adding a clinic creates a schema and initial access, not a new clinic database.

Company onboarding sequence

- A signed-in platform operator submits a clinic ID, metadata, time zone and initial administrator details. The platform records a durable registration reservation.
- The API creates/prepares the target schema inside appointments, checks it is not already bound to another clinic, applies clinic migrations and creates the initial administrator.
- The registry marks the clinic Ready and refreshes API routing. Only eligible ready clinics enter public discovery.
- Registration states are Provisioning, Ready and Failed. Failure preserves records/schema for diagnosis. Retrying the same ID with the same registration details resumes safely; it does not silently overwrite an existing clinic.

Schema evolution has two tracks

Track	Current source
Clinic EF migrations	InitialPostgres → ServiceCatalogue → AppointmentHistory → PatientProfiles. Migration history is separate inside each clinic schema.
Platform registry SQL	005-platform-registry.sql creates company/registry tables; 006-shared-clinic-registry.sql adapts registry metadata to shared clinic storage.
Shared extension preparation	prepare-shared-clinic-database.sql handles btree_gist in careline_extensions; extension ownership may require PostgreSQL administrator execution.

The published API includes the platform SQL assets. The --migrate maintenance switch initializes/applies supported migrations and exits; --clinic <id> restricts the clinic migration selection. Platform initialization is still part of the maintenance startup path. Review configuration and a backup before running it against operational data.

DEPLOYMENT IS NOT A RESET

Migrations update structure while preserving supported data. Do not use destructive reset scripts or remove schemas as a routine release step. Recent readability, availability-preview and concurrency changes use the existing data model and require no new migration.

12 Hosting, configuration and operations

Source control, deployment and the database are separate parts of operating the system.

The described test installation serves Web from 10.20.1.100:5544, with a separately published API and PostgreSQL at 10.20.1.102:5432. These are deployment details supplied during the project, not live connectivity verified by this guide. Check IIS bindings and the Web API URL for the actual running installation.

Setting / identity	What to retain securely
Web: Backend:Mode / Backend:ApiBaseUrl	Api mode and the actual API base URL. Web has no clinic database connection string.
API: ConnectionStrings:Appointments	Shared clinic database host/name and runtime database login, commonly careline_app.
API: ConnectionStrings:Platform	Optional company database connection, commonly platform_owner. Keep it separate from clinic data.
Tenancy / Platform configuration	Clinic IDs, names, time zones, discovery/registration flags and platform bootstrap lifecycle.
Hosting settings	AllowedHosts, TLS/proxy bindings, IIS application pools, Demo disabled, controlled migration settings and persistent data-protection keys.
Administrative access	PostgreSQL postgres administration is distinct from runtime access. Store passwords and recovery procedures in a password manager, not in this document or Git.

Release and recovery checklist

- Pull the approved source, build Release, publish the components affected by the release, and include Web wwwroot and API database assets. Preserve server-specific secrets and configuration.
- Back up both operational databases before structural changes. Apply required migrations once under a controlled maintenance procedure; restart and verify sign-in, booking, conflicts and public discovery.
- Schedule whole-database backups, protect key material and document a tested restore procedure. Keep platform registry and clinic bindings consistent across restores.
- Monitor application/database failures, storage, backups and authentication problems. A healthy process endpoint alone does not prove a user workflow works.

13 Verification, limitations and growth

The code is tested in isolated environments; test evidence is not a certificate of production readiness.

What the test layers prove

Layer	What is exercised
Core executable tests	Scheduling rules, role boundaries, time/DST rules, history, agreed times and status transitions.
PostgreSQL / API integration	Real transactions and constraints, multiple API instances, sessions, competing writes, migration preservation and persistence.
Tenant / platform tests	Schema routing, cross-clinic denial, onboarding, global email checks and public/private projections.
MVC / DOM workflows	Forms, redirects, antiforgery, searchable controls, version conflicts and role-specific actions.
Chromium browser checks	Rendered responsive screens, keyboard/touch interactions, layout overflow and public journeys.

For the latest readability/public-availability change, validation included a clean Release build, 89 PostgreSQL platform/discovery checks, 112 Chromium checks, 45 MVC contract checks and 85 DOM workflow checks. A separate browser run against the real API verified anonymous free-time browsing and no-schedule messages without JavaScript. Counts describe those local runs, not tests of the live IIS installation.

Known boundaries to understand

- Patient identity/membership is clinic-specific; discovery across clinics does not create shared patient access.
- Email/SMS reminders, MFA, verified recovery and external calendar/EHR connections remain future integrations.
- The workspace bootstrap and some administrative lists are not fully paged. Large clinic datasets need windowed/paginated UI loading before scale claims.
- Multi-API database safety is implemented. Multiple Web hosts additionally require shared protected cookie keys; rate limiting needs a deployment-wide policy.
- Background updates use polling; reports use current state and estimated historical capacity. Neither is real-time streaming or a historical schedule ledger.

14 Glossary and source map

Use this page when handing the project to another developer or support team.

Term	Plain-language meaning
Tenant	One clinic with its own scoped records and users.
Schema / search path	A namespace of PostgreSQL tables / the server-selected lookup order for those tables.
Transaction	A group of writes that commits together or rolls back together.
Advisory lock	A database-coordinated agreement that makes competing application operations wait.
Exclusion constraint	A database rule rejecting overlapping occupied intervals for a doctor or patient.
Optimistic concurrency	Save only if the record’s version still matches the form that was opened.
Idempotency / request ID	Recognize a repeated intended booking and return its original result without creating a duplicate.
DTO / migration	An explicit public/shared data message / a versioned change to database structure.
UTC / local clinic time	An unambiguous stored instant / the wall-clock date and time displayed to clinic users.

Where the implementation lives

Topic	Repository source
Routing, platform, public preview	src/Appointments.Api/Services/ClinicRegistry.cs; SharedClinicDatabase.cs; PlatformService.cs; PublicDiscovery.cs
Rules and lifecycle	src/Appointments.Core/AppointmentService.cs; SchedulingRules.cs; SchedulingClock.cs; Catalogue.cs
Storage and concurrency	src/Appointments.Data/AppointmentsDbContext.cs; PostgresAppointments.cs; Migrations/
Browser and API boundaries	src/Appointments.Web/Services/ApiClient.cs; Controllers/; Views/; wwwroot/js/app.js; src/Appointments.Contracts/
Setup and feature details	docs/SHARED_DATABASE.md; TERMINO_BOOKING.md; APPOINTMENT_HISTORY.md; REPORTS_AND_CALENDAR.md
Verification and deployment	tests/; .github/workflows/; src/Appointments.Api/Program.cs; src/Appointments.Web/Program.cs

Repository: github.com/RisteLalkov/AppointmentsWEB. This guide follows current code and newer shared-database/booking documents where older notes describe superseded per-clinic database or protected-only availability designs. Recheck source and update this guide after architecture changes.